

Anvesh Chandra Reddy 192472303

ITA0502

Computer Vision

Slot -C

Constraint Based Problem Solving

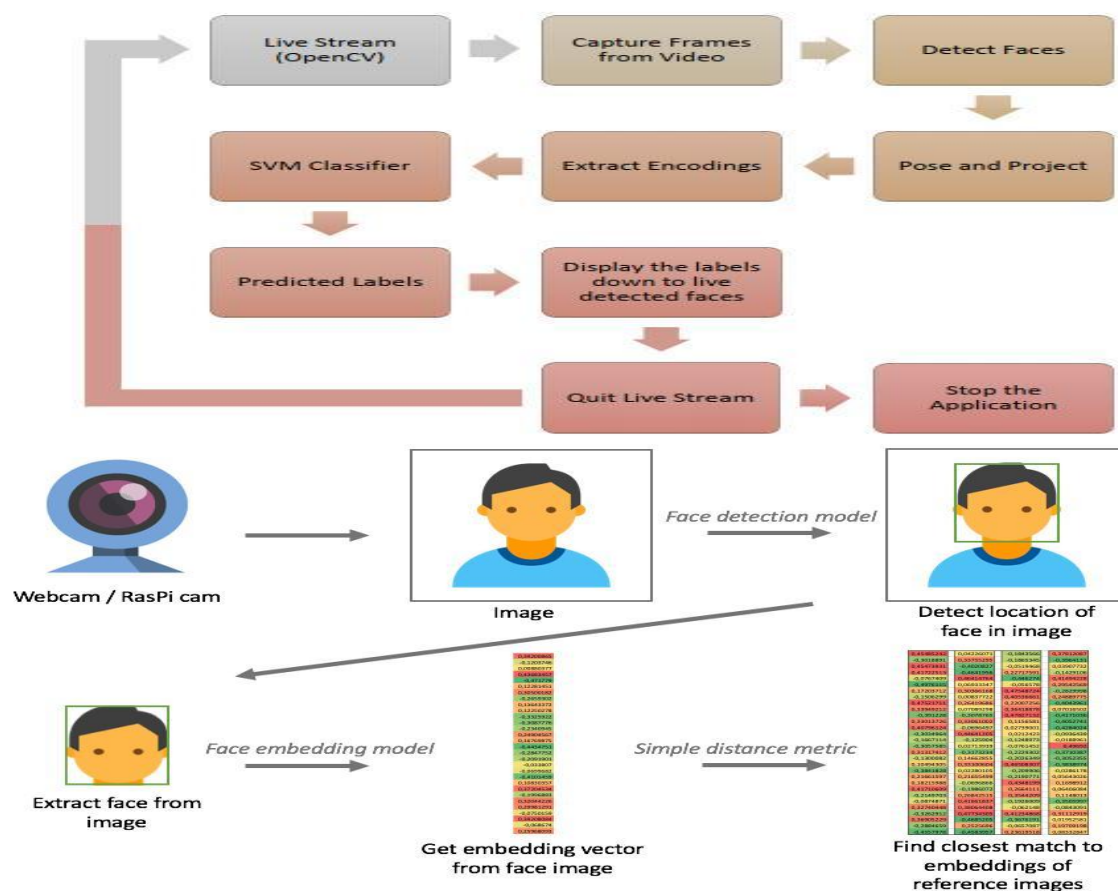
1. Real-Time Face Recognition for Campus Security

Problem

The university needs a low-cost, real-time system that can detect and recognize registered students and staff from CCTV while handling different lighting conditions and minimizing false recognition.

Proposed OpenCV Pipeline

Live Camera → Preprocessing → Face Detection → Face Alignment → Feature Extraction → Face Recognition → Confidence Check → Identification



Step 1: Image Acquisition

- Capture live frames using an OpenCV camera interface such as VideoCapture.
- Process frames continuously from the CCTV stream.
- Resize frames to a suitable resolution to reduce processing time.

Step 2: Preprocessing

The captured image is improved before recognition.

Operations include:

- Convert image to grayscale.
- Apply histogram equalization or CLAHE to improve uneven lighting.
- Apply Gaussian/median filtering to reduce camera noise.
- Resize detected face regions to a fixed size.

Step 3: Face Detection

OpenCV can use a Haar Cascade classifier or a more accurate DNN-based face detector.

The detector identifies the location of faces using bounding boxes.

Step 4: Face Alignment

The detected face is normalized so that differences in head position have less effect.

For example:

- Align eyes approximately horizontally.
- Crop the face region.
- Resize it to a standard size.

Step 5: Feature Extraction

For a low-cost OpenCV system, LBPH (Local Binary Pattern Histogram) can be used.

LBPH extracts local texture patterns from the face and is relatively lightweight.

For a more robust system, facial embeddings from a deep face-recognition model can be used.

Step 6: Recognition

Compare the extracted facial features with the registered student/staff database.

Example:

Detected Face



Extract Features



Compare with Database

↓
 Similarity/Confidence Score
 ↓
 Registered? → Yes → Identify Person
 → No → Unknown Person

Step 7: Optimization for Real-Time Performance

To operate on a low-cost computer:

- Resize input frames.
- Process every 2nd or 3rd frame if necessary.
- Detect faces periodically rather than on every frame.
- Track already detected faces between detection frames.
- Use lightweight models such as LBPH where appropriate.
- Avoid unnecessary image processing.
- Use grayscale images for computationally expensive stages.

Final Design

Stage	Technique
Acquisition	CCTV / OpenCV VideoCapture
Preprocessing	Grayscale + CLAHE + Filtering
Detection	Haar Cascade / OpenCV DNN
Alignment	Face cropping and normalization
Feature Extraction	LBPH / Facial Embeddings
Recognition	Database comparison
Verification	Confidence threshold

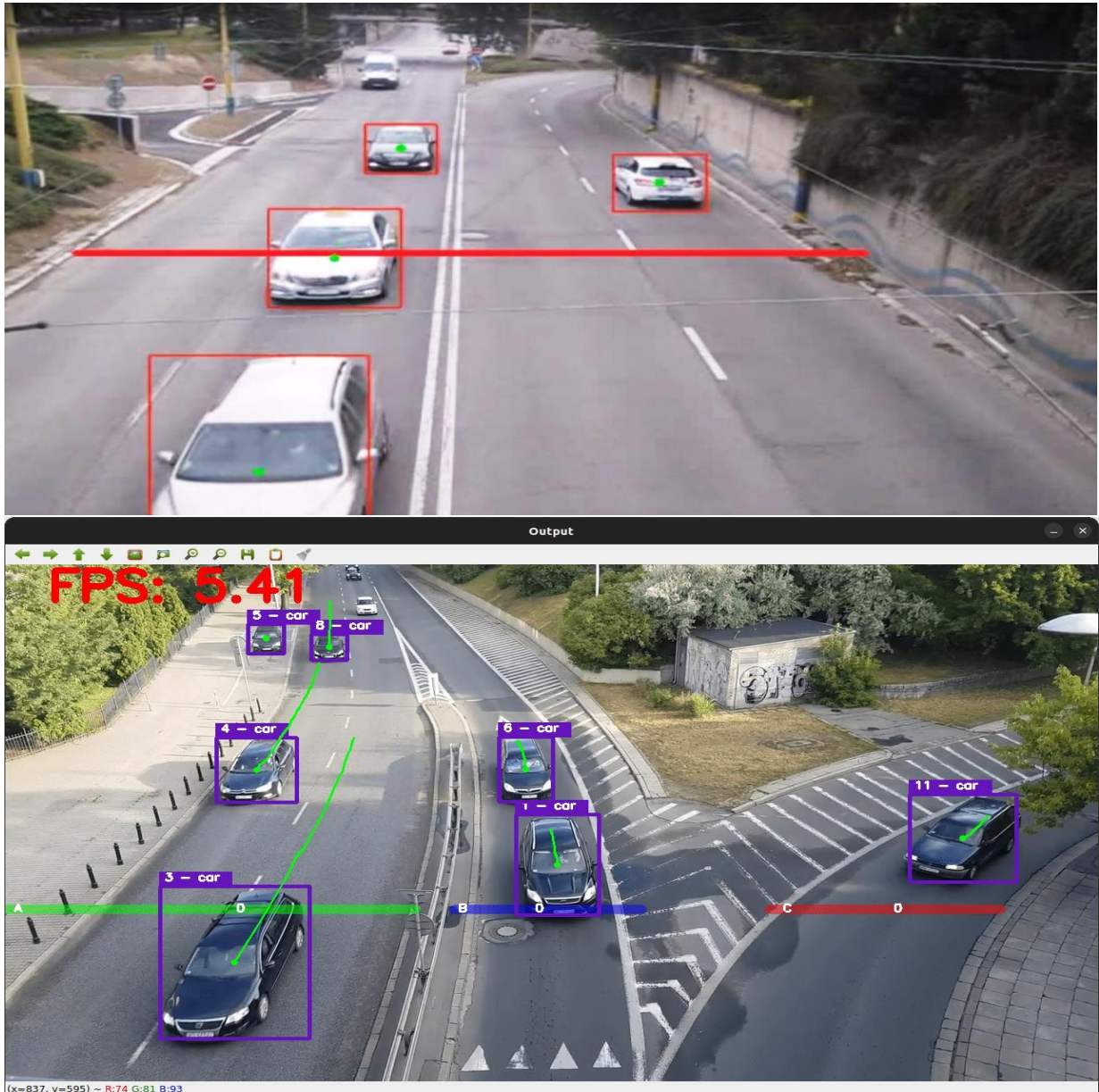
2. Intelligent Traffic Violation Detection System

Problem

The traffic system must detect, track, count, and analyze vehicles from roadside cameras while minimizing errors caused by shadows, noise, dynamic backgrounds, and multiple vehicles.

Proposed OpenCV Pipeline

Video Acquisition → Preprocessing → Vehicle Detection → Motion Analysis → Tracking → Line Crossing → Violation Detection → Counting → Visualization



Step 1: Video Acquisition

- Capture live traffic video using OpenCV.
- Read frames continuously.
- Define the Region of Interest (ROI) containing the road.

Step 2: Preprocessing

To improve detection:

- Resize frames.
- Convert to grayscale when appropriate.
- Apply Gaussian filtering to reduce camera noise.
- Improve contrast if lighting is poor.

Step 3: Vehicle Detection

Vehicles can be detected using:

- Background subtraction for a simple fixed-camera system.
- Contour detection after foreground extraction.
- OpenCV DNN/object detection for more reliable vehicle classification.

Step 4: Motion Analysis

For a fixed roadside camera, background subtraction can identify moving regions.

A foreground mask is generated:

Current Frame



Background Model



Difference



Foreground Mask

Step 5: Morphological Processing

The foreground mask may contain small holes and noise.

Use:

- Opening → removes small noise.
- Closing → fills small gaps.
- Dilation → strengthens vehicle regions.

Step 6: Vehicle Tracking

Each detected vehicle receives a temporary ID.

Example:

Vehicle 1 → Frame 1 → Frame 2 → Frame 3 → ID 1

Vehicle 2 → Frame 1 → Frame 2 → Frame 3 → ID 2

Tracking prevents the same vehicle from being counted repeatedly.

Step 7: Line Crossing

Draw a virtual line across the road.

When the center point of a tracked vehicle moves from one side of the line to the other:

Vehicle



Before Line



Crosses Line



Count = Count + 1

A vehicle ID should only trigger the count once.

Step 8: Abnormal Movement Detection

Track vehicle trajectories.

Possible abnormal behavior:

- Sudden direction change.
- Wrong-way movement.
- Stopping in restricted areas.
- Unusual lane movement.
- Sudden reversal.

Handling Common Problems

Problem	Solution
Shadows	Shadow suppression + HSV/color analysis
Camera noise	Gaussian/median filtering
Holes in vehicles	Morphological closing
Multiple vehicles	Object IDs + tracking
Double counting	Count each tracking ID once
Camera vibration	Stabilization or fixed-camera calibration
Illumination changes	Adaptive threshold/background model

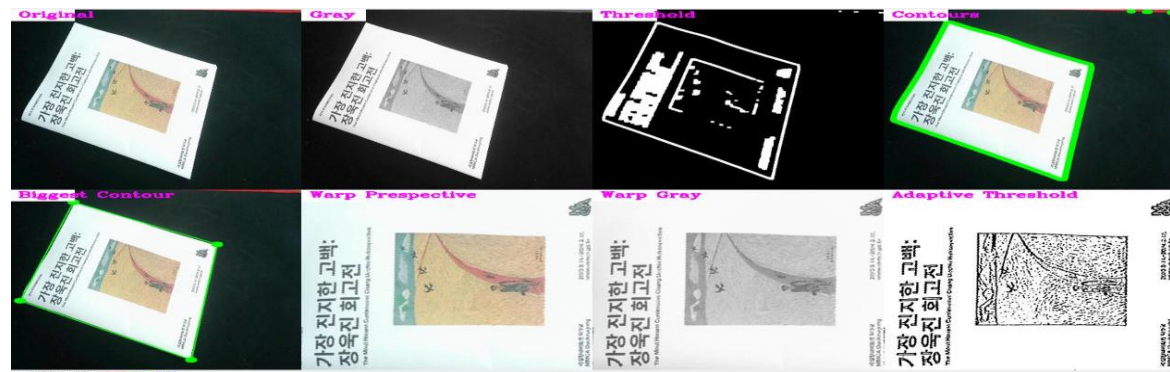
3. Automated Document Processing System

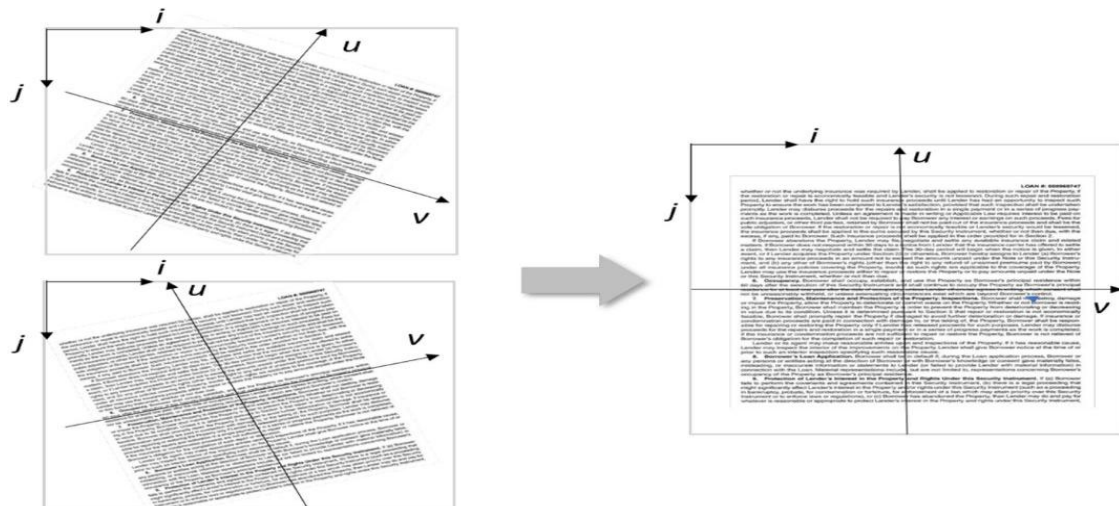
Problem

The company receives thousands of documents containing blur, noise, uneven illumination, different font sizes, and poor-quality scans. The goal is to improve them before OCR.

Proposed OpenCV Pipeline

Input Document → Grayscale → Noise Removal → Illumination Correction → Thresholding → Morphological Processing → Geometric Correction → Text Segmentation → OCR





Step 1: Image Acquisition

- Load scanned documents using OpenCV.
- Convert different input formats into a consistent image representation.
- Resize very large images when necessary.

Step 2: Grayscale Conversion

Convert the color document into grayscale.

This reduces three color channels to one intensity channel and simplifies further processing.

Step 3: Noise Removal

Use:

- Gaussian filter for general noise.
- Median filter for salt-and-pepper noise.

This makes characters smoother and easier for OCR to recognize.

Step 4: Illumination Correction

Uneven lighting can cause some areas to be darker or brighter.

Use techniques such as:

- Background estimation.
- Adaptive thresholding.
- CLAHE.

This helps produce more uniform text regions.

Step 5: Thresholding

Convert the grayscale image into a binary image.

For uneven illumination, adaptive thresholding is particularly useful.

Grayscale Image



Adaptive Threshold



Text + Background Separation

Step 6: Morphological Operations

Use morphological operations to improve character shapes.

Opening:.

Erosion:

- Removes small unwanted regions and can refine thick characters.

Step 7: Geometric Correction

Scanned documents may be rotated or skewed.

Perform:

- Deskewing.
- Perspective correction.
- Rotation correction.

This ensures that text lines are properly aligned before OCR.

Step 8: Text Segmentation

Separate the document into:

- Paragraphs.
- Lines.
- Words.
- Text blocks.

Contours and connected-component analysis can help identify text regions.

Step 9: OCR

The cleaned and segmented image is passed to an OCR engine such as Tesseract.

Clean Document



Text Regions



OCR



Extracted Text

Step 10: Batch Processing

For thousands of documents:

- Process documents automatically from folders.
- Log failed documents for later review.

Role of Each Stage

Stage	Main Purpose	OCR Benefit
Grayscale	Simplify image	Easier processing
Filtering	Remove noise	Cleaner characters
Illumination correction	Normalize brightness	Better text visibility
Thresholding	Separate text/background	Clear characters
Morphology	Repair/refine characters	Better character shapes
Segmentation	Isolate text regions	Easier OCR
Geometric correction	Fix skew/perspective	Correct text orientation

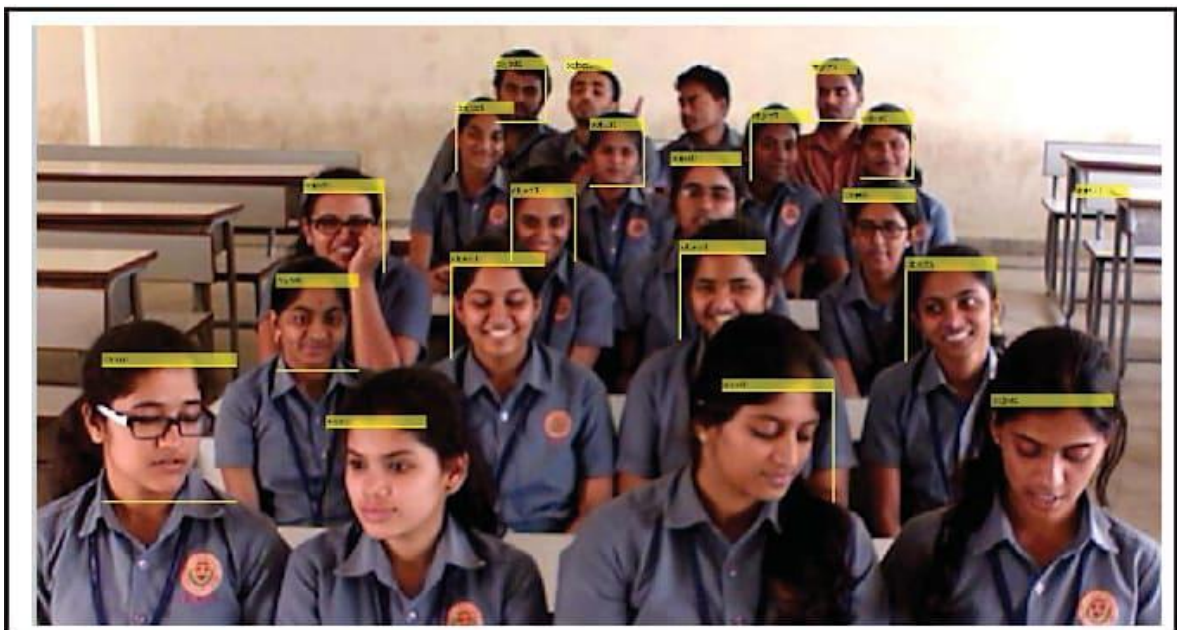
4. Real-Time Multi-Person Tracking and Attendance System

Problem

The college needs to identify multiple students from classroom CCTV, track them across frames, and record attendance only once per student.

Proposed OpenCV Pipeline

CCTV → Preprocessing → Multi-Face Detection → Feature Extraction → Recognition → Tracking → Identity Management → Attendance Verification → Database





(a) Face Detection using TB-MTCNN

(b) Face Detection and Tracking using TB-MTCNN with Deep SORT

Step 1: Video Acquisition

Capture classroom CCTV frames continuously using OpenCV.

Step 2: Preprocessing

For each frame:

- Resize the frame.
- Convert to grayscale where appropriate.
- Improve lighting using histogram equalization/CLAHE.
- Apply filtering to reduce camera noise.

Step 3: Multiple Face Detection

Detect all faces in each frame.

Example:

Frame

↓

Face 1 → Student A

Face 2 → Student B

Face 3 → Student C

Face 4 → Unknown

A Haar Cascade can be used for a lightweight implementation, while an OpenCV DNN face detector provides stronger robustness.

Step 4: Face Feature Extraction

For every detected face:

- Crop the face.
- Align and resize it.
- Extract facial features.

For a lightweight system, LBPH can be used. For higher recognition reliability, facial embeddings are preferable.

Step 5: Recognition

Compare each detected face with the registered student database.

Each face receives:

- Student ID/name, or
- Unknown if confidence is insufficient.

Step 6: Tracking

Detection does not need to happen on every frame.

After detecting a student, track the person's location between detection frames.

Possible tracking approaches include:

- CSRT/KCF trackers for individual objects.
- Multi-object tracking using detected bounding boxes and identity association.

Maintaining Identity

The system maintains a record such as:

Track ID	Student	Status
T01	Student A	Present
T02	Student B	Present
T03	Student C	Present
T04	Unknown	Not marked

When the same person moves slightly, the tracking ID remains associated with the recognized identity.

Instead of immediately deleting the identity:

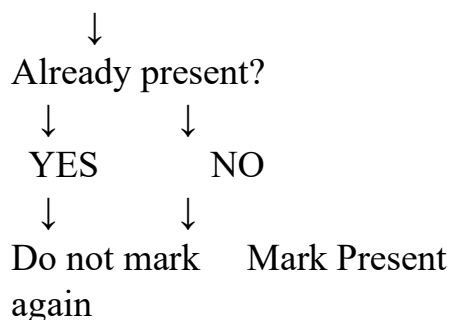
- Keep the tracking ID for a short period.

Preventing Duplicate Attendance

Maintain a set/database of students already marked present.

Example:

Student A detected



Therefore, even if Student A appears in hundreds of frames, attendance is recorded only once.

False Recognition Prevention

- Use a confidence threshold.
- Use lightweight recognition methods when hardware is limited.
- Avoid processing duplicate frames unnecessarily.

5. Smart Home Surveillance and Intrusion Detection

Problem

A fixed camera must detect significant movement, distinguish real intrusions from background changes, track objects, and generate an alert when an unauthorized person enters a predefined region.

Proposed OpenCV Pipeline

Camera → Background Modeling → Foreground Detection → Filtering → Morphology → Contour Detection → Object Tracking → ROI Analysis → Intrusion Decision → Alert





Step 1: Video Acquisition

Use a fixed camera and continuously capture frames with OpenCV.

A fixed camera is useful because the background remains relatively stable.

Step 2: Background Subtraction

Create a background model and compare each new frame against it.

Current Frame



Compare with Background



Foreground / Motion Mask

Suitable OpenCV approaches include:

- MOG2.
- KNN background subtraction.

Step 3: Filtering

The motion mask can contain small noise caused by:

- Camera sensor noise.
- Small moving objects.
- Leaves.
- Lighting variations.

Apply Gaussian or median filtering to reduce noise.

Step 4: Morphological Operations

Use:

Opening

- Removes small isolated noise.

Closing

- Fills small gaps inside moving objects.

Dilation

- Connects nearby foreground pixels and strengthens detected objects.

Step 5: Contour Detection

Find contours in the cleaned foreground mask.

Very small contours are discarded using an area threshold.

For example:

Contour Area < Minimum Area



Ignore

This prevents tiny movements from triggering alarms.

Step 6: Object Detection and Tracking

Track significant moving objects using:

Step 8: Suspicious Activity Detection

The system can analyze:

- Object entering the entrance ROI.
- Direction of movement.
- Duration of presence.
- Object size.
- Movement trajectory.

A combination of these conditions can reduce false alarms.

Handling False Alarms

Problem

Camera noise

Small background changes

Shadows

Leaves

Sudden brightness change

Repeated detection

Non-human movement

Solution

Gaussian/median filtering

Minimum contour-area threshold

Shadow detection/suppression

Remove small contours

Background-model adaptation

Object tracking

Combine motion with object/person detection

Final Decision Logic

Motion Detected?



YES



Large Enough Object?



YES

